

RL-PQC-VPN ❓❓❓ 3-Month Team Delivery Plan

RL-PQC-VPN — 3-Month Team Delivery Plan

Goal for these 3 months: take the existing Phase 1 (hybrid PQC crypto) and Phase 2 (RL agent + Layer 1 anomaly detection) work — both already implemented and tested — to a shipped, working **desktop application**, backed by a hardened self-hosted WireGuard+PQC server, with a fully trained and evaluated RL agent and complete 3-layer anomaly detection.

Mobile is explicitly out of scope for these 3 months — no mobile UI, no app store builds, no phone testing. What *is* in scope: making every architectural decision in a way that a mobile app can be added later by writing a thin native shell around the same core, instead of a rewrite. That constraint shapes Member 1’s work in particular (Section 3).

1. Team & Ownership

Member	Owns	Why this split
Member 1	Rust core, Tauri desktop shell, end-to-end integration	Needs to touch every subsystem to wire it together — natural integration-lead role
Member 2	WireGuard + PQC handshake server, deployment	Server-side is self-contained and platform-agnostic — same server serves desktop today and mobile later with zero changes if designed right
Member 3	RL agent (training, evaluation) + anomaly detection (Layers 2 & 3)	These two are tightly coupled — the anomaly <code>threat_score</code> is a direct input to the RL agent’s state vector, so one person owning both avoids an interface getting redesigned twice

2. Decisions That Must Be Frozen in Week 1 (all three members, one meeting)

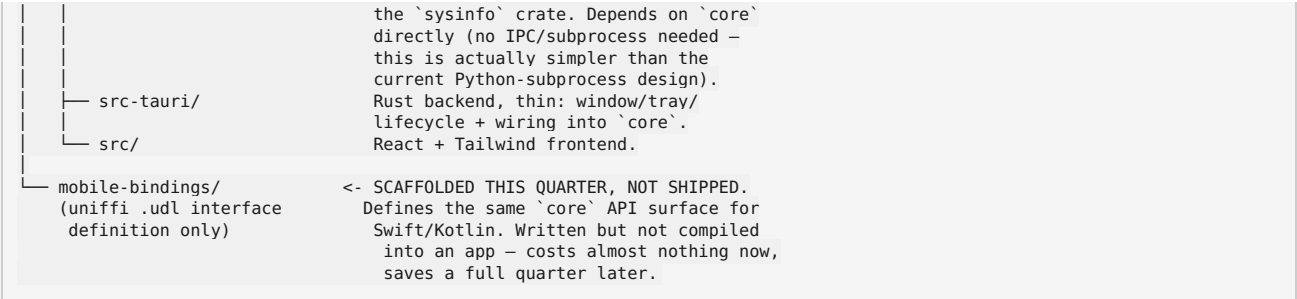
Everything downstream depends on these three contracts *not* changing mid-project. Get them wrong and weeks 6-12 turn into rework.

- Handshake wire protocol** (Member 2 proposes, all review): the exact message format for the client↔server PQC handshake exchange — independent of what platform the client runs on. This is what makes “mobile-ready server” true on day one instead of a promise.
- RL state/action interface** (Member 3 proposes, all review): already exists and is stable — 7-dim normalized state vector, 4-action space sourced from `algo_registry.py`. Freezing it means Member 1 can build the Rust-side consumer of this interface without waiting on Member 3’s training work to finish.
- Rust workspace layout** (Member 1 proposes, all review): see Section 3 — this is the decision that determines whether mobile is a future option or a future rewrite.

3. Keeping the Mobile Option Open (without building it)

The current Python implementation cannot become a mobile app directly — iOS does not allow a persistent background Python process at all, and Android’s equivalent is similarly restrictive. The fix is architectural, not something to solve later: **port the core logic to Rust now, structured as a workspace that separates “shared logic” from “platform glue” from day one.**

pqc-vpn/ ├─ core/ │ ├── crypto/ │ ├── anomaly/ │ ├── rl/ │ └── state/ └─ protocol/ └─ desktop/	(Cargo workspace) <- ALL shared logic lives here, zero platform-specific code allowed in (hybrid KEM, ML-DSA-65 auth, key store) (Layers 1-3, CPU-gated combiner) (ONNX inference over the trained policy) (defines a `DeviceState` trait — platforms implement it, core never reads psutil/sysinfo/etc. directly) (client-side of the wire protocol frozen in Section 2) <- Tauri app. Implements `DeviceState` using
---	--



3	Port <code>key_store.py</code> (secure wipe semantics). Define the <code>DeviceState</code> and <code>TunnelHandle</code> traits core will depend on.
4	Basic Tauri shell: window, system tray, connect/disconnect button wired to a stub. Checkpoint: architecture contracts frozen (Section 2) — sync with Members 2 & 3.
5	Implement <code>desktop/</code> 's <code>DeviceState</code> using the <code>sysinfo</code> crate (CPU/RAM/network — the Rust equivalent of <code>state_observer.py</code>).
6	Integrate <code>protocol/</code> client side against Member 2's server (may still be a stub/mock server at this point — coordinate). First real handshake over the network, Rust client ↔ Rust-or-mock server.
7	Integrate Member 3's exported ONNX policy via the <code>ort</code> crate. Wire <code>rl/</code> output into <code>crypto/</code> 's algorithm selection.
8	Integrate Member 3's anomaly <code>threat_score</code> combiner into the state pipeline. Checkpoint: first full vertical slice — real desktop app, real agent decision, real handshake, against Member 2's real (not mock) server.
9	WireGuard PSK injection (<code>wg set</code> wrapped from Rust) — tunnel actually comes up end to end.
10	React dashboard: live connection status, chosen algorithm, threat layers active, rekey countdown — reading directly from <code>core/</code> in-process (no WebSocket needed on desktop, unlike the original FastAPI-based proposal design — simpler now that there's no subprocess boundary).
11	Packaging (<code>.deb</code> , AppImage via <code>cargo tauri build</code>), bug-fixing from integration testing with both other members.
12	Final polish, mobile-readiness review (confirm <code>core/</code> has zero platform leakage — this is the checkpoint that actually validates Section 3's promise), demo prep.

Member 2 — WireGuard + PQC Server (Desktop-Serving, Mobile-Ready)

Week	Work
1	Provision the VPS (DigitalOcean/AWS Lightsail, Ubuntu). Install WireGuard. Draft the wire protocol proposal for Week 1's sync (Section 2).
2	Checkpoint: protocol frozen. Implement the server-side PQC handshake responder (<code>liboqs</code>) matching Member 1's client-side implementation — same algorithm set, same ML-DSA-65 server identity/signing logic.
3	Server-side <code>wg set</code> PSK injection for a single test peer — get one manual end-to-end handshake working against a throwaway client script (doesn't need to wait on Member 1's Tauri app).
4	Containerize the handshake responder (Docker). Checkpoint: sync on protocol stability with Member 1.
5	Multi-peer session management — the server needs to track which WireGuard peer maps to which active PQC session, since production use means more than one client.
6	Support live rekey: when a connected client's agent triggers <code>rekey-now</code> , the server must accept a fresh handshake for an <i>existing</i> peer and swap its PSK without dropping the tunnel. First real integration test against Member 1's client.
7	Replay protection and basic handshake rate-limiting (a PQC handshake responder exposed on the internet is a target — this matters even for a coursework-stage deployment).
8	Checkpoint: first full vertical slice , working alongside Members 1 & 3.
9	Session logging (connect/disconnect/rekey events) — needed for the dashboard and for Member 3's evaluation work (baseline

	comparisons need session-level data).
10	Load/soak testing — repeated connect/disconnect/rekey cycles, confirm no leaked state or crashed sessions.
11	Deployment automation (Docker Compose, or CI-driven redeploy), write up the wire protocol as a versioned spec document — this is the single artifact that makes “mobile-ready” a checkable claim rather than an assertion, since it proves the server makes no assumption about client platform.
12	Hardening pass based on integration bugs found by the other two members, demo prep, support final evaluation runs.

Member 3 — RL Agent & Anomaly Detection

Week	Work
1	Pick up from the existing smoke-tested agent (see <code>PROGRESS.md</code> / <code>PROJECT_BRIEFING.md</code> Section 6 for the two reward bugs already found and fixed). Start addressing the known weak point: the ML-KEM-768/1024 decision boundary is too narrow under realistic CPU load.
2	Longer training runs + an entropy/exploration bonus targeted at under-visited actions (ML-KEM-1024 currently almost never gets selected — confirmed via brute-force reward-grid analysis to be optimal in only ~1.3% of the state space, so it needs deliberate oversampling during training, not just more steps).
3	Re-verify policy behavior the same way it was verified before (bucket-by-security-need distribution check, jitter-stability check) — confirm all four actions now have a <i>robust</i> margin, not just a technically-correct-on-average one.
4	Export the trained policy to ONNX. Verify numerically that ONNX inference matches PyTorch inference on a batch of test states (this parity check matters — a silent export bug would be invisible until it showed up as weird behavior on-device). Checkpoint: sync interface with Member 1.
5	Build anomaly detection Layer 2 (rule-based signatures): port scan detection, retransmission-spike detection, MitM-latency-signature detection, bandwidth-spike/exfiltration detection, DNS-server-change detection. Each as an independent stateful check over the rolling 5-second metric window.
6	Wire Layer 2 into the CPU-gated combiner (< 70% CPU) alongside the existing always-on Layer 1. Unit tests per signature (inject a synthetic port-scan pattern, confirm it fires; inject normal traffic, confirm it doesn’t).
7	Build anomaly detection Layer 3 (Isolation Forest): define the feature vector (packet size distribution, rate, inter-arrival time, directionality), collect/simulate benign-traffic training data, train <code>sklearn.IsolationForest</code> .
8	Export the Isolation Forest for Rust consumption (via <code>skl2onnx</code> , or a from-scratch Rust port — Isolation Forest inference is just decision-tree traversal, cheap to reimplement if the ONNX export path proves awkward). Wire into the combiner with the < 40% CPU gate. Checkpoint: first full vertical slice — all three anomaly layers plus the RL agent feeding real decisions into Member 1’s client.
9	Design and run the baseline evaluation the proposal itself calls for: trained agent vs. classical WireGuard (no PQC), vs. static ML-KEM-768, vs. a simple rule-based policy — across at least three defined scenarios (idle, elevated threat, resource-constrained).
10	Collect evaluation metrics: connection latency overhead, CPU usage, and effective security level per baseline. Write up results.
11	Tuning pass based on evaluation findings; support integration bug-fixing with Members 1 & 2.

12	Final evaluation report, demo prep — be ready to explain the full reward-design story (Section 6 of <code>PROJECT_BRIEFING.md</code>) since it's the most substantive technical narrative in this whole track.
----	---

6. Cross-Cutting Integration Checkpoints

These are the points where all three members need to actually sit down together, not just work in parallel:

- **End of Week 1:** contracts frozen (Section 2) — nothing after this point should require renegotiating the wire protocol, the state/action interface, or the workspace layout.
- **End of Week 4:** everyone re-confirms the contracts held up under actual implementation (it's normal for something to need a small amendment here — better to catch it now than at Week 10).
- **End of Week 6/8:** first full vertical slice — a real (if unpolished) desktop app, connecting to the real server, using the real trained agent and real anomaly layers. This is the point to demo internally before showing anything to a supervisor as “working.”
- **End of Week 10:** feature-complete; remaining time is hardening, evaluation, and polish — not new integration work.
- **End of Week 12:** final demo, final evaluation report, mobile-readiness review.

7. Key Risks

Risk	Mitigation
Member 1 blocked on Member 3's ONNX export (Week 4) or Member 2's protocol (Week 2)	Both are scheduled to land <i>before</i> Member 1 needs them (Weeks 2 and 4 respectively); if either slips, Member 1 works against a mock/stub in the meantime rather than stalling
core/ accumulates platform-specific shortcuts under deadline pressure, quietly breaking the mobile-readiness promise	Explicit Week 12 review checking for trait-boundary violations, not just “does it compile”
RL agent's narrow decision margins (Section 5, Member 3 Weeks 1-3) aren't fully fixed by Week 4	Ship with the current smoke-tested model if needed and keep improving in parallel — Member 1 doesn't need the <i>final</i> model, just a stable ONNX interface, which can be re-exported later without touching client code
Server hardening (rate limiting, replay protection) gets deprioritized under integration pressure	Scheduled explicitly in Weeks 7 and not shared with a feature-integration week, so it doesn't silently get cut

8. What This Plan Deliberately Does Not Include

- The mobile app itself (UI, app store builds, phone-specific testing) — only the architectural groundwork that makes it a future option.
- Windows support — the proposal's own timeline already scopes this as a later phase.
- A production-grade multi-region server deployment — one VPS is enough to prove the architecture and run the evaluation.

These are natural candidates for “what's next” in the same conversation where this plan gets presented — worth having ready as a follow-up answer, not omitting by accident.